

# KnowOps no Obsidian: Uma Arquitetura Local para Conhecimento Pessoal Compilado, Operacional e Agentivo

**Autor:** Paulo Tomazinho, PhD

**Organização:** CKF Research / KnowOps

**Data:** Maio de 2026

---

## Resumo

Este artigo apresenta a visão e a implementação inicial de um sistema **Personal KnowOps** sobre o Obsidian, usando o **Compiled Knowledge Format (CKF)** como camada de compilação semântica. O Obsidian é utilizado como ambiente humano de autoria, leitura, navegação e manutenção de notas em Markdown. O CKF atua como compilador de conhecimento, transformando notas humanas em pacotes estruturados, rastreáveis e consumíveis por agentes. O KnowOps surge como a camada operacional que transforma uma coleção de notas em um sistema vivo de conhecimento: capaz de responder perguntas, detectar relações semânticas, gerar unidades atômicas, preservar provenance, criar QA pairs, manter histórico semântico e alimentar agentes. A implementação inicial validou um fluxo local no Windows: uma nota Markdown no vault Obsidian é enviada por script Node.js ao CKF MCP, compilada em `.ckf.json` e salva como sidecar dentro do próprio vault. A partir desse MVP, propomos a evolução para um plugin nativo Obsidian com comandos como `Compile current note`, `Compile folder`, `Compile vault`, `View CKF graph`, `Semantic links`, `Semantic diff`, `QA generation` e `Agent chat`. A tese central é que Obsidian, CKF e KnowOps representam três camadas complementares: Obsidian é a interface humana, CKF é a representação intermediária compilada, e KnowOps é o runtime operacional de conhecimento.

**Palavras-chave:** KnowOps; CKF; Obsidian; Personal Knowledge Management; compilação de conhecimento; agentes de IA; Markdown; MCP; semantic memory; personal knowledge operations; retrieval; provenance.

---

## 1. Introdução

Ferramentas de Personal Knowledge Management, como Obsidian, Notion, Roam, Logseq e wikis pessoais, resolveram uma parte importante do problema contemporâneo do conhecimento: permitiram que indivíduos criassem, conectassem e mantivessem redes de notas. Elas transformaram documentos soltos em vaults, páginas, backlinks, tags e grafos navegáveis.

Mas uma limitação permanece: grande parte desse conhecimento continua **passivo**.

Ele está escrito, mas não opera.

Está conectado por links, mas nem sempre por relações semânticas.

Está salvo, mas não necessariamente é recuperável no momento certo.

Está disponível para leitura humana, mas não estruturado para raciocínio agnóstico.  
Está versionado como texto, mas não como conhecimento.

O conceito de **KnowOps** nasce dessa lacuna.

Assim como DevOps operacionaliza software, MLOps operacionaliza modelos e DataOps operacionaliza dados, **KnowOps operacionaliza conhecimento**.

KnowOps pergunta:

Como fazer conhecimento pessoal e organizacional deixar de ser arquivo e passar a ser infraestrutura operacional?

A resposta proposta neste artigo é combinar três elementos:

1. **Obsidian** como ambiente local de escrita e navegação humana;
2. **CKF** como compilador de conhecimento estruturado;
3. **MCP / agentes** como ponte entre conhecimento compilado e uso operacional.

Essa combinação transforma uma pasta de arquivos Markdown em uma base pessoal de conhecimento compilado, auditável, interrogável e agnóstica.

---

## 2. A visão: de PKM para Personal KnowOps

O modelo clássico de PKM é centrado na nota.

nota → tag → link → backlink → graph

Esse modelo é poderoso para humanos. Ele permite escrever, navegar, lembrar e conectar ideias. Mas para agentes e LLMs, tags e links ainda são sinais rasos.

Uma tag diz:

#RAG

Um wikilink diz:

[[CKF]]

Mas nenhum dos dois explicita se a relação é:

CKF complementa RAG  
CKF não substitui RAG  
CKF muda o substrate de retrieval

CKF reduz composition hallucination sob retrieval pressure  
CKF depende de source traceability

KnowOps propõe passar de links documentais para unidades operacionais de conhecimento.

O novo fluxo é:

```
nota Markdown
→ compilação CKF
→ unidades semânticas
→ relações operacionais
→ retrieval agentivo
→ decisões, perguntas, alertas e workflows
```

A nota continua existindo. O humano continua escrevendo em Markdown. Mas, em paralelo, cada nota passa a ter uma versão compilada em CKF.

---

## 3. A tese das três camadas

A arquitetura KnowOps no Obsidian pode ser entendida em três camadas.

### 3.1 Obsidian como Human Knowledge IDE

O Obsidian é o ambiente humano.

Ele oferece:

- Markdown;
- pastas;
- backlinks;
- tags;
- wikilinks;
- graph view;
- plugins;
- vault local;
- controle do usuário sobre os arquivos.

No KnowOps, o Obsidian não é substituído. Ele continua sendo o lugar onde o conhecimento é escrito, editado e revisado.

Sua função é:

```
interface humana de autoria e curadoria
```

### 3.2 CKF como camada compilada

O CKF é a representação intermediária.

Ele transforma texto humano em:

- entidades;
- conceitos;
- princípios;
- regras de decisão;
- procedimentos;
- exceções;
- anti-patterns;
- QA pairs;
- retrieval chunks;
- atomic units;
- source traceability.

Sua função é:

transformar prosa em conhecimento operacional tipado

### 3.3 KnowOps como runtime operacional

KnowOps é a camada que usa o conhecimento compilado.

Ela responde perguntas como:

- Quais decisões esta nota suporta?
- Quais regras existem neste vault?
- Que conceitos dependem de outros?
- Que exceções contradizem uma regra geral?
- O que mudou semanticamente desde a última versão?
- Quais notas estão sem rastreabilidade?
- Que perguntas este conhecimento deveria responder?
- Que conhecimento precisa de revisão humana?

Sua função é:

fazer conhecimento operar continuamente

---

## 4. A arquitetura geral

A arquitetura proposta é:

```
Obsidian Vault
  ↓
Markdown notes
  ↓
CKF Compiler via MCP
  ↓
.ckf.json / .ckf.md / .ckf.yaml sidecars
  ↓
Knowledge index
  ↓
Semantic links, QA, diff, retrieval, agent chat
  ↓
Personal KnowOps Runtime
```

Em termos práticos, dentro do vault:

```
Meus Livros/
  10-Notes/
    Minha primeira nota CKF.md
    Teste pequeno.md

  40-CKF/
    json/
      Minha primeira nota CKF.ckf.json
      Teste pequeno.ckf.json
    markdown/
    yaml/
    debug/
```

O Markdown é a fonte humana.

O `.ckf.json` é o sidecar compilado.

O `.debug.json` registra a resposta bruta do MCP para auditoria.

---

## 5. Implementação inicial validada

A implementação inicial já demonstrou o ciclo mínimo funcional.

### 5.1 Ambiente

O ambiente usado foi:

```
Windows
Obsidian instalado localmente
```

```
Vault: Meus Livros
Node.js v24.16.0
CKF MCP endpoint público
Script local compile-note.js
```

## 5.2 Fluxo validado

O fluxo foi:

1. Criar nota Markdown no Obsidian
2. Rodar script Node.js local
3. Script lê o arquivo .md
4. Script envia conteúdo ao CKF MCP
5. MCP retorna pacote CKF
6. Script salva .ckf.json em 40-CKF/json
7. Obsidian mantém a nota humana; filesystem mantém o sidecar CKF

O comando validado foi equivalente a:

```
node "C:\Users\Paulo Tomazinho\Documents\Scripts\compile-note.js" "C:\Users\Paulo Tomazinho\Documents\Meus Livros Obsidian\Meus Livros\10-Notes\Teste pequeno.md" "C:\Users\Paulo Tomazinho\Documents\Meus Livros Obsidian\Meus Livros\40-CKF\json\Teste pequeno.ckf.json"
```

O resultado foi a criação de arquivos como:

```
Teste pequeno.ckf.json
Teste pequeno.ckf.json.debug.json
Minha primeira nota CKF.ckf.json
Minha primeira nota CKF.ckf.json.debug.json
```

Essa etapa prova a viabilidade do núcleo:

```
Obsidian Markdown → CKF MCP → sidecar CKF local
```

---

## 6. O papel do sidecar CKF

O sidecar é a ponte entre nota humana e conhecimento agentivo.

Para cada nota:

```
Minha nota.md
Minha nota.ckf.json
```

O `.md` responde:

Como o humano escreveu isso?

O `.ckf.json` responde:

Que conhecimento estruturado existe aqui?

Essa separação é essencial.

Ela evita corromper a experiência humana de escrita e, ao mesmo tempo, cria uma camada operacional para agentes.

---

## 7. Do script ao plugin nativo

O script local foi importante para validar o fluxo. Mas ele ainda exige comandos manuais e caminhos longos.

O próximo salto arquitetural é um plugin Obsidian nativo:

```
CKF Personal KnowOps Plugin
```

O plugin substituirá:

```
terminal + script + caminhos manuais
```

por comandos internos do Obsidian:

```
CKF: Compile current note
CKF: Compile folder
CKF: Compile vault
CKF: Open CKF sidecar
CKF: Show package summary
```

---

## 8. MVP do plugin Obsidian CKF

O MVP deve ser deliberadamente simples.

## 8.1 Comandos iniciais

```
CKF: Compile current note
CKF: Open CKF sidecar
CKF: Show package summary
```

## 8.2 Settings

O plugin deve ter configurações:

```
ckfEndpoint = https://compiledknowledgeformat.org/api/mcp
outputFolder = 40-CKF
defaultFormat = json
saveDebugResponse = true
compileOnSave = false
debounceSeconds = 30
ignoredFolders = ["40-CKF", ".obsidian"]
```

## 8.3 Fluxo do comando “Compile current note”

1. Obter nota ativa
2. Verificar se é Markdown
3. Ler conteúdo
4. Enviar ao CKF MCP
5. Extrair package CKF
6. Salvar em 40-CKF/json/<note>.ckf.json
7. Salvar debug em 40-CKF/debug/<note>.debug.json
8. Atualizar frontmatter
9. Mostrar Notice de sucesso

## 8.4 Frontmatter sugerido

Após compilar, a nota pode receber:

```
ckf_status: compiled
ckf_package: 40-CKF/json/Minha nota.ckf.json
ckf_compiled_at: 2026-05-23T17:55:11Z
ckf_compiler: ckf.compile
ckf_protocol: ckf-1.0
```

Isso permite identificar notas compiladas, staleness e versão do pacote.



## 9. Código conceitual do plugin

O núcleo do plugin em TypeScript seria:

```
const file = this.app.workspace.getActiveFile();

if (!file || file.extension !== ".md") {
  new Notice("Abra uma nota Markdown primeiro.");
  return;
}

const markdown = await this.app.vault.cachedRead(file);

const response = await fetch(this.settings.ckfEndpoint, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    jsonrpc: "2.0",
    id: Date.now(),
    method: "tools/call",
    params: {
      name: "ckf.compile",
      arguments: {
        text: markdown,
        format: "json"
      }
    }
  })
});

const raw = await response.text();
const json = JSON.parse(raw);
const ckf = json.result.content[0].text;

await this.app.vault.adapter.write(
  `40-CKF/json/${file.basename}.ckf.json`,
  ckf
);

await this.app.vault.adapter.write(
  `40-CKF/debug/${file.basename}.debug.json`,
  raw
);

new Notice(`CKF compilado: ${file.basename}`);
```

Esse é o núcleo mínimo. O restante é ergonomia, painel, settings e tratamento de erro.

## 10. Do CKF sidecar ao KnowOps index

Depois que cada nota tem um `.ckf.json`, o sistema pode construir um índice global.

Esse índice pode agregar:

- todos os conceitos;
- todas as entidades;
- todas as regras;
- todos os procedimentos;
- todos os retrieval chunks;
- todos os QA pairs;
- todas as source traceability entries.

Exemplo:

```
40-CKF/index/  
  concepts.index.json  
  entities.index.json  
  retrieval.index.json  
  qa.index.json  
  graph.index.json
```

Isso permite que o vault deixe de ser uma coleção de arquivos e vire uma base operacional.

---

## 11. Semantic links

Backlinks tradicionais são sintáticos:

```
Nota A linka Nota B
```

Semantic links são relacionais:

```
Conceito A depends_on Conceito B  
Regra X has_exception Exceção Y  
Princípio P supports Decisão D  
Claim C contradicts Claim D  
Procedimento Z implements Regra R
```

O plugin pode gerar uma tabela ou arquivo:

```
40-CKF/index/semantic-links.json
```

Com relações:

```
{
  "source": "unit_001",
  "target": "unit_045",
  "relation_type": "is_exception_to",
  "confidence": 0.91,
  "source_note": "Política de despesas.md",
  "target_note": "Aprovações.md"
}
```

Essa camada é o começo do grafo KnowOps.

---

## 12. Semantic diff

O Obsidian já permite histórico de arquivo por plugins, sync ou Git. Mas o KnowOps precisa de algo diferente:

O texto mudou ou o conhecimento mudou?

Duas versões podem ter texto diferente e significado igual. Ou podem ter texto parecido e uma exceção crítica nova.

O semantic diff compara pacotes CKF:

Minha nota v1.ckf.json  
Minha nota v2.ckf.json

E responde:

- unidades adicionadas;
- unidades removidas;
- regras alteradas;
- exceções novas;
- procedimentos modificados;
- conceitos renomeados;
- claims contraditos.

Esse recurso transforma versionamento textual em versionamento cognitivo.

---

## 13. QA generation

Cada pacote CKF pode gerar perguntas.

Exemplo:

```
Q: O que é CKF?
Q: O que CKF não substitui?
Q: Qual a diferença entre visual fidelity e operational fidelity?
Q: Quando usar CKF em vez de texto bruto?
Q: Quais exceções se aplicam a esta regra?
```

Essas perguntas servem para:

- estudo;
- revisão;
- flashcards;
- avaliação de compreensão;
- chat agentivo;
- detecção de lacunas.

No Obsidian, o plugin pode criar:

```
40-CKF/qa/Minha nota.qa.md
```

Ou uma visualização lateral com QA pairs.

---

## 14. Agent chat

O estágio mais poderoso é o chat com o vault.

Mas o chat não deve usar apenas Markdown bruto. Ele deve usar:

```
retrieval_chunks
atomic_units
qa_pairs
procedures
exceptions
source_traceability
```

Fluxo:

```
pergunta do usuário
→ busca em retrieval_chunks
→ complementa com atomic_units
→ verifica exceptions/procedures
```

- responde com source excerpts
- aponta notas originais

Isso reduz o risco de o agente apenas “interpretar” uma nota longa. Ele passa a operar sobre conhecimento compilado.

---

## 15. Compile folder e compile vault

Depois do MVP, o plugin deve compilar em escala.

### Compile folder

Seleciona pasta → compila todos os .md daquela pasta → salva sidecars

Útil para:

- um projeto;
- um curso;
- um livro;
- uma área temática.

### Compile vault

Compila todo o vault, exceto pastas ignoradas

Mas deve respeitar:

- hash de conteúdo;
- staleness;
- debounce;
- fila de compilação;
- limite de chamadas;
- retry;
- logs.

---

## 16. Staleness e recompilação

Cada nota deve saber se seu CKF está atualizado.

Critério simples:

```
content_hash atual != content_hash compilado  
→ status = stale
```

Frontmatter:

```
ckf_status: stale  
ckf_last_hash: abc123
```

Ou índice global:

```
{  
  "note": "Minha nota.md",  
  "ckf_status": "stale",  
  "last_compiled_at": "...",  
  "reason": "content_hash_changed"  
}
```

Isso permite um comando:

```
CKF: Compile stale notes
```

---

## 17. Review queue

Como LLM extraction pode errar, o KnowOps precisa de revisão.

O plugin pode marcar para revisão:

- baixa confiança;
- source excerpt ausente;
- language mismatch;
- truncation;
- possível contradição;
- regra sem exceção;
- procedimento incompleto;
- entidade ambígua.

Isso gera:

```
40-CKF/review/review-queue.md
```

Com itens como:

- [ ] Revisar claim: "CKF substitui RAG" – possível contradição
- [ ] Revisar procedure sem steps
- [ ] Revisar conceito com baixa confiança

## 18. A visão de produto

O produto final não é apenas um plugin técnico.

É uma nova experiência:

Obsidian como IDE de conhecimento humano  
CKF como compilador semântico  
KnowOps como runtime operacional  
Agentes como operadores do conhecimento

O usuário escreve normalmente. O sistema compila em segundo plano. O conhecimento passa a ficar disponível para:

- perguntas;
- decisões;
- alertas;
- revisão;
- síntese;
- conexão semântica;
- aprendizado;
- planejamento;
- automação.

## 19. Diferença entre PKM e KnowOps

| PKM tradicional   | KnowOps                     |
|-------------------|-----------------------------|
| notas             | unidades de conhecimento    |
| tags              | entidades/conceitos tipados |
| backlinks         | relações semânticas         |
| busca textual     | retrieval chunks            |
| histórico textual | semantic diff               |
| leitura humana    | operação agentiva           |
| graph visual      | graph operacional           |

| PKM tradicional | KnowOps            |
|-----------------|--------------------|
| memória passiva | memória executável |

A diferença central:

PKM ajuda o humano a lembrar. KnowOps ajuda humanos e agentes a operar conhecimento.

## 20. Roadmap sugerido

### Fase 0 — Script local validado

Status: concluído.

Markdown → CKF MCP → .ckf.json

### Fase 1 — Plugin MVP

- Compile current note;
- salvar `.ckf.json`;
- salvar debug;
- atualizar frontmatter;
- mostrar summary.

### Fase 2 — Compilação em escala

- Compile folder;
- Compile vault;
- compile stale notes;
- content hash;
- queue.

### Fase 3 — Visualização semântica

- CKF side panel;
- package summary;
- concepts/entities browser;
- retrieval chunks viewer;
- source traceability viewer.

### Fase 4 — KnowOps intelligence

- semantic links;
- semantic diff;
- QA generation;



- review queue;
- contradiction detection.

## Fase 5 — Agent runtime

- chat com vault;
  - respostas com provenance;
  - execução de procedures;
  - alertas;
  - workflows;
  - integração MCP.
- 

## 21. Critérios de sucesso

Um MVP bem-sucedido deve permitir:

1. abrir uma nota Markdown;
2. clicar em `CKF: Compile current note`;
3. gerar `.ckf.json` em `40-CKF/json`;
4. salvar debug em `40-CKF/debug`;
5. atualizar frontmatter;
6. mostrar contagens do pacote;
7. abrir o sidecar;
8. repetir sem quebrar o vault;
9. detectar se a nota está stale;
10. compilar outra nota com o mesmo fluxo.

Um produto maduro deve permitir:

1. compilar vault inteiro;
  2. gerar índice CKF global;
  3. consultar conhecimento por agente;
  4. detectar relações semânticas;
  5. comparar versões semanticamente;
  6. revisar itens de baixa confiança;
  7. responder com provenance;
  8. operar como memória pessoal persistente.
- 

## 22. Riscos e decisões de design

### 22.1 Não misturar Markdown e CKF

O Markdown deve continuar limpo. O CKF deve ficar como sidecar.

## 22.2 Não recompilar tudo a cada save

Usar debounce, hash e fila.

## 22.3 Não confiar cegamente no LLM

Usar confidence, source excerpts, review queue e debug.

## 22.4 Não esconder provenance

Toda resposta agentiva deve poder apontar para a nota e o trecho original.

## 22.5 Não tratar graph como fim

O graph é uma visualização. A unidade central é conhecimento operacional.

---

# 23. Conclusão

A implementação de KnowOps no Obsidian é uma consequência natural da evolução do CKF.

O CKF provou que documentos humanos podem ser compilados em pacotes estruturados e rastreáveis. O Obsidian oferece um ambiente local, popular e extensível para escrita e organização. KnowOps une os dois como runtime operacional.

O resultado é uma nova arquitetura:

```
Obsidian = Human Knowledge IDE
CKF = Knowledge Intermediate Representation
KnowOps = Operational Knowledge Runtime
MCP/Agents = Execution and Interaction Layer
```

O primeiro MVP já foi validado via script local. Uma nota Markdown foi compilada pelo CKF MCP e salva como `.ckf.json` dentro do vault. Esse pequeno resultado é arquiteturalmente importante: ele mostra que um vault Obsidian pode se tornar uma wiki compilável.

O próximo passo é transformar o script em plugin nativo. A partir daí, o sistema deixa de ser um experimento manual e passa a ser uma plataforma pessoal de conhecimento operacional.

A visão final é clara:

O futuro do conhecimento pessoal não é apenas escrever melhor notas. É compilar, operar e evoluir conhecimento com agentes.

Esse é o papel do KnowOps no Obsidian.

---

## 24. Apêndice — Prompt para Lovable criar o plugin MVP

Crie um plugin Obsidian chamado CKF Personal KnowOps.

Objetivo:

Transformar um vault Obsidian em um sistema Personal KnowOps usando CKF.

Funcionalidade MVP:

1. Comando ``CKF: Compile current note``.
2. Ler a nota Markdown ativa.
3. Enviar o conteúdo para ``https://compiledknowledgeformat.org/api/mcp`` usando JSON-RPC.
4. Chamar ferramenta ``ckf.compile`` com ``{ text, format: "json" }``.
5. Salvar o resultado em ``40-CKF/json/<note>.ckf.json``.
6. Salvar resposta bruta em ``40-CKF/debug/<note>.debug.json``.
7. Atualizar frontmatter da nota com status CKF.
8. Mostrar Notice de sucesso ou erro.
9. Criar settings para endpoint, output folder, save debug e compile on save.
10. Criar comando ``CKF: Show package summary`` com contagens principais.

Use TypeScript e APIs oficiais do Obsidian.

Não usar backend próprio.

Preparar arquitetura para futuras funções: compile folder, compile vault, semantic diff, semantic links, QA generation e agent chat.